

Tema 1: Classes i orientació a objectes

1.1 Repàs: structs i pas de paràmetres

Una **struct** agrupa diverses dades relacionades.

```
cpp [main.cpp] struct Rellotge {      int h, m, s; }; // Cal posar el ';'
```

Pas de paràmetres a funcions

- **Per valor:** Es fa una còpia. Lent i no modifica l'original.
- **Per referència (&):**
 - **Lectura (const + &):** Ràpid i segur. No hi ha còpia. `void mostrar(const Rellotge& r);`
 - **Escriptura (&):** Modifica l'original directament. `void avançar(Rellotge& r);`

1.2 Tipus abstractes de dades (TAD) i classes

Un **TAD** agrupa dades (atributs) i les operacions (mètodes) per manipular-les. En C++, s'implementa amb **classes** per garantir la consistència de les dades.

Encapsulació

Blinda les dades per evitar modificacions incontrolades externes: - **private:** Atributs ocults al codi extern. Només accessibles pels mètodes de la mateixa classe. - **public:** Mètodes accessibles des de fora. Són la interfície o única via vàlida per interactuar amb l'objecte.

Organització en fitxers

Separem el “què fa” del “com ho fa”.

1. Especificació (.hpp o .hh)

Defineix l'estructura de la classe. Inclou guàrdies (**#ifndef**) per evitar incusions múltiples i bucles de compilació.

```
“cpp [Punt.hpp] #ifndef PUNT_HPP #define PUNT_HPP

class Punt { double x, y; // Privat per defecte

public: Punt(); // Constructor per defecte Punt(double a, double b); // Constructor amb paràmetres
Punt(const Punt& altre); // Constructor de còpia

void moure(double dx, double dy); // Modificador
double get_x() const;           // Consultor (const = no modifica)

}; #endif
```

2. Implementació (.cpp` o `.cc`)

Conté el codi real de les funcions declarades.

- `**`#include "Punt.hpp"`**`: Carrega les definicions de dades prèvies.
- `**Operador `::`**` (Resolució d'àmbit): Indica a quina classe pertany la funció (ex: `Punt::moure``).

```
```cpp [Punt.cpp]
#include "Punt.hpp"

Punt::Punt() { x = 0; y = 0; }
Punt::Punt(double a, double b) { x = a; y = b; }
Punt::Punt(const Punt& altre) { x = altre.x; y = altre.y; }

void Punt::moure(double dx, double dy) {
 x += dx; y += dy; // Accés als atributs de l'objecte actual
}

double Punt::get_x() const { return x; }
```

### 3. Ús (main.cc)

Creació i ús d'objectes.

```
“`cpp [main.cpp] #include #include “Punt.hpp”
```

```
int main() { Punt p(1, 2); // Crida al constructor amb paràmetres p.moure(3, 3); // ‘p’ actua com a paràmetre implícit
```

```
// std::cout << p.x; // ERROR! 'x' és privat
std::cout << p.get_x(); // Correcte
}
```

`*[Simulació interactiva disponible a la web]*`

---

#### ## 1.3 Conceptes addicionals de classes

- `**Paràmetre implícit (`this`)**`: És un punter intern amagat que referencia l'objecte sobre el qual crida.
- `**Mètodes `inline`**`: S'escriuen directament dins de l'`hpp`. El compilador substitueix la crida d'aquesta funció.
- `**Membres `static`**`:
  - `*Atributs*`: Es comparteixen globalment entre tots els objectes de la mateixa classe (ex: un comptador).
  - `*Mètodes*`: No requereixen instanciar cap objecte per executar-se. Es criden mitjançant la referència a la classe.

#### ### Genericitat (`template`)

Permeten crear estructures i funcions intel·ligents indiferents cap al tipus de dada que emmagatzemen (o processen).

```
```cpp
template <class T>
class Capsa {
    T contingut;
public:
    Capsa(T x) { contingut = x; }
};
// Ús: Capsa<int> enter(5); Capsa<string> text("Hola");
```

1.4 Compilació amb Makefile

Per gestionar projectes eficientment amb molts fitxers (`main.cc`, `Punt.hpp`, `Punt.cpp`), deleguem la feina a un fitxer de comandes **Makefile**. Una vegada programat, executar simplement **make** unificarà els codis `.o` només si han patit canvis recents.

```
““makefile [Makefile] CXX = g++ CXXFLAGS = -Wall -std=c++17
program: main.o Punt.o $(CXX) -o program main.o Punt.o
main.o: main.cc Punt.hpp $(CXX) $(CXXFLAGS) -c main.cc ““
```

1.5 Per si no t'agrada llegir teoria

Aquest simulador conté un projecte sencer amb classes, mètodes, `this`, `inline`, `static` i un **Makefile**.

[Simulació interactiva disponible a la web]